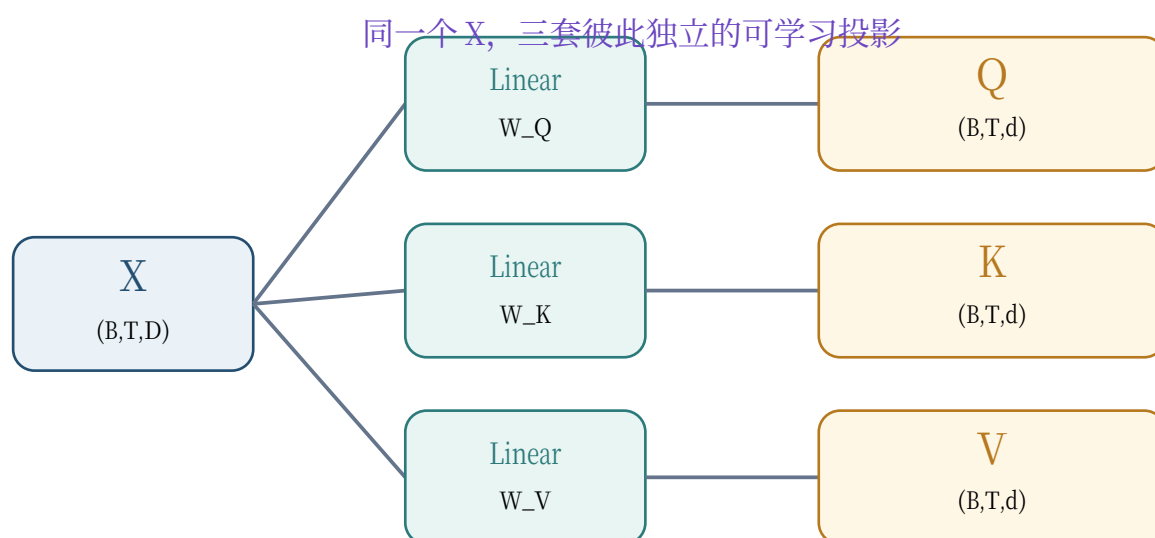


第 53 讲

Q、K、V 是怎样算出来的

从 X 到 $Q = X W_Q$ 、 $K = X W_K$ 、 $V = X W_V$

本讲定位	上一讲已经把 Token ID 变成了 $X:(B,T,D)$ 。这一讲只解决一个问题：同一个 X 为什么要经过三套不同的 Linear Projection，变成 Q、K、V？注意：到本讲结束时，我们还没有真正做 Attention。
------	--



本讲必须真正掌握	验收标准
Q/K/V 的来源	能从 X 和三套 Weight 写出 Shape 与数值计算
Projection 的本质	知道它仍然只是 Linear / Matrix Multiplication
Token 是否混合	知道 QKV projection 本身不会让 Token 彼此交流
PyTorch 对应	能解释 nn.Linear 的 weight 为什么是 (out,in)
工程实现	知道三套 Linear 可以融合成一次 Linear(D,3D)

1. 先把上一讲的 X 找回来

上一讲中，Token ID 经过 Embedding Table 查表以后，得到每个 Token 的 D 维向量。把一批序列放在一起，最常见 Shape 是：

X.shape = (B, T, D_model)

其中 B 是 batch size，T 是 sequence length，D_model 是每个 Token 当前表示的特征维数。X[b,t,:] 就是一条样本中第 t 个 Token 的 D_model 维表示。

先别把 X 想复杂

进入 Attention 以前，X 只是一堆 Token 向量。Q、K、V 并不是从外部额外拿来的数据，而是同一批 X 经过不同可学习矩阵重新投影得到的三种表示。

例如一个极小序列有 3 个 Token，每个 Token 4 维：

X.shape = (3,4)

Token	X 的 4 个特征
Token 1	[1, 0, 1, 0]
Token 2	[0, 1, 0, 1]
Token 3	[1, 1, 0, 0]

后面我们将用同一组数字贯穿整讲，所以每一步都能手算，而不是只看抽象公式。

2. Q、K、V 到底各自扮演什么角色

Attention 想解决的核心问题是：对于当前 Token，我应该从其他 Token 读取多少信息？要完成这件事，模型把同一个 Token 表示拆成三种“角色”。

符号	英文	直觉角色	后面真正做什么
Q	Query	我现在想找什么	与各 Key 做匹配，产生 score
K	Key	我具有什么可匹配的特征	被 Query 比较
V	Value	如果你关注我，我实际提供什么内容	按 Attention 权重被加权汇总
通俗比喻		把 Attention 想成检索系统：Query 像搜索条件，Key 像索引标签，Value 像真正要取回的内容。但这只是帮助理解角色，Q/K/V 本质上仍然都是数字向量。	
重要边界		Q 不是“问题文本”、K 不是“关键词字符串”、V 也不是“原始 Token 内容”。它们都是通过可学习 Linear Projection 产生的连续向量表示。	

为什么不能直接拿 X 同时承担三种角色？因为模型希望“匹配空间”和“被读取内容空间”拥有不同的可学习变换。三套独立 Weight 就给了它这种自由度。

3. Q/K/V 的公式其实仍然只是 Linear Projection

$$Q = X @ W_Q \quad K = X @ W_K \quad V = X @ W_V$$

如果暂时忽略 bias， 设：

对象	Shape	含义
X	(B,T,D_model)	每个 Token 当前的 D_model 维表示
W_Q	(D_model,d_q)	把 X 投影到 Query 空间
W_K	(D_model,d_k)	把 X 投影到 Key 空间
W_V	(D_model,d_v)	把 X 投影到 Value 空间
Q	(B,T,d_q)	每个 Token 的 Query
K	(B,T,d_k)	每个 Token 的 Key
V	(B,T,d_v)	每个 Token 的 Value

矩阵乘法只作用在最后一个特征维：

$$(B,T,D_model) @ (D_model,d_q) \rightarrow (B,T,d_q)$$

和第 5/6/28 讲的连接	这并没有引入新的数学。你已经学过 $XW+b$ 和 <code>nn.Linear</code> 。Q/K/V 只是把同样的 Linear 运算同时做三次，而且三次使用不同 Parameter。
----------------	---

4. 用具体数字手算一个 Query

取第二个 Token：

$$x_2 = [0, 1, 0, 1]$$

Query 投影矩阵：

W_Q	第1列	第2列	第3列
第1个输入特征	1	0	1
第2个输入特征	0	1	1
第3个输入特征	1	1	0
第4个输入特征	0	1	-1

于是第一个 Query 分量是 x_2 与 W_Q 第一列做 Dot Product：

$$q_2[0] = 0*1 + 1*0 + 0*1 + 1*0 = 0$$

$$q_2[1] = 0*0 + 1*1 + 0*1 + 1*1 = 2$$

$$q_2[2] = 0*1 + 1*1 + 0*0 + 1*(-1) = 0$$

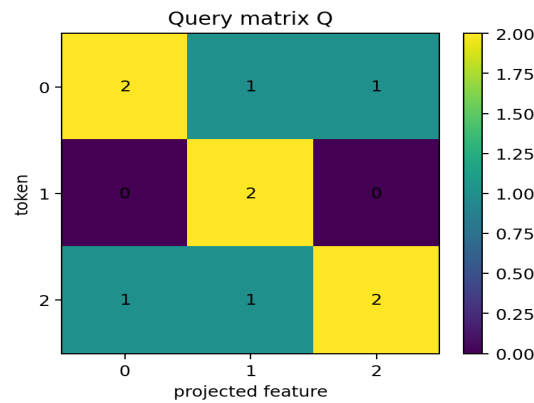
$$q_2 = [0, 2, 0]$$

底层本质	一个 Token 的 Query 向量里，每一个输出数字，都只是“这个 Token 的输入向量”和 W_Q 的某一列做一次 Dot Product”。
------	---

5. 把 3 个 Token 一次矩阵乘完

矩阵乘法可以把逐 Token 的手算一次完成。实验中的结果是：

Token	Q	K	V
1	[2,1,1]	[1,2,1]	[1,0,3]
2	[0,2,0]	[2,-1,1]	[1,3,1]
3	[1,1,2]	[2,1,1]	[1,2,0]

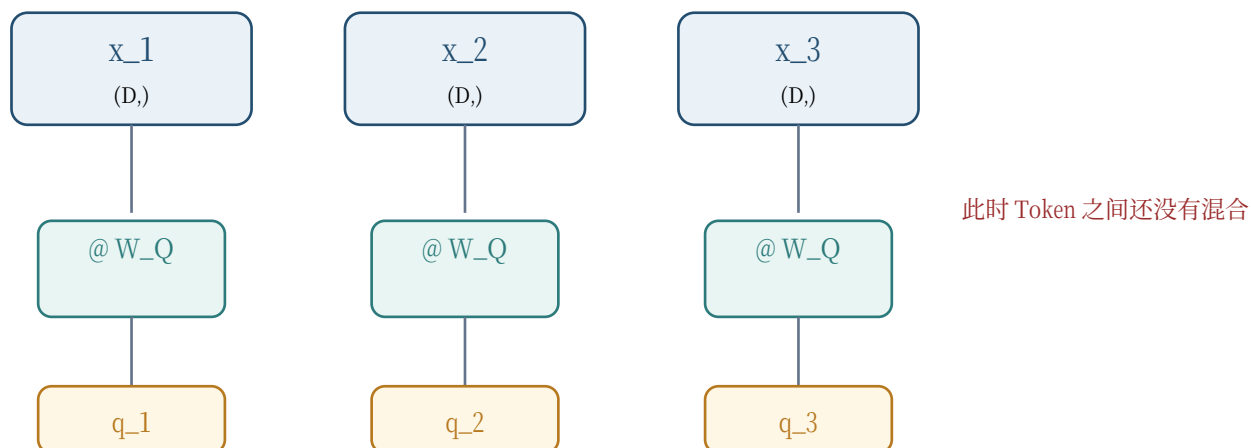


Q、K、V 数值不同，不是因为输入 Token 不同，而是因为它们使用了不同的投影矩阵。即使来自同一个 X，三种角色也可以学习完全不同的特征组合。

一个很重要的反事实	如果强行令 $W_Q = W_K = W_V$ ，并且 bias 也相同，那么 $Q=K=V$ 。Attention 仍然可以运行，但模型失去了三种角色分工的自由度。
-----------	---

6. Projection 阶段并不会让 Token 彼此交流

Projection 阶段：每个 Token 独立使用同一套 W_Q



对固定 W_Q 来说：

$$q_t = x_t @ W_Q$$

q_1 只依赖 x_1 ； q_2 只依赖 x_2 。此时还没有 q_1 读取 x_2 的操作。K 和 V 同理。

实验中我们只把 Token 1 的第一个输入特征增加 10，再重新计算 Q。结果只有第 1 行 Q 改变，Token 2 和 Token 3 的 Q 完全不变。

为什么这一点重要

很多初学者看到 Q/K/V 就以为 Attention 已经发生。实际上 QKV Projection 只是“为 Attention 准备三套表示”。真正的 Token-to-Token mixing 要等到下一讲计算 QK^T 、Softmax 和加权 V。

7. Batch 进入以后，Shape 仍然遵守同一条规则

X: (B,T,D_model)
W_Q: (D_model,d_q)
Q: (B,T,d_q)

例如实验构造：

X3.shape = (2,3,4)

经过三套 4 -> 3 Projection 后：

Q3.shape = K3.shape = V3.shape = (2,3,3)
--

B 和 T 都保留，只有最后一个特征维从 4 变成 3。这与第 6 讲中 Linear 处理 (B,T,D) 的规律完全相同。

维度	Projection 前	Projection 后	是否改变
Batch	B	B	否
Sequence	T	T	否
Feature	D_model	d_q / d_k / d_v	可能改变
Shape 阅读习惯	以后读 Transformer 代码时，先不要看变量名，先追 Shape。Q/K/V 仍然是一批 Token 向量，只是最后一个 feature dimension 被不同矩阵重新映射。		

8. PyTorch nn.Linear 为什么要转置 Weight

数学上我们习惯写：

$$Q = X @ W_Q, W_Q.shape = (D_{in}, D_{out})$$

但 PyTorch：

```
q_proj = nn.Linear(D_in, D_out, bias=False)
print(q_proj.weight.shape)
# -> (D_out, D_in)
```

因此 PyTorch 前向等价于：

$$Q = X @ weight^T + bias$$

实验里 NumPy 的 W_Q 是 (4,3)，复制给 PyTorch 时使用 $W_Q.T$ ，因此 $q_proj.weight.shape=(3,4)$ 。最终 Q 的最大误差为 0。

视角	Weight Shape	Forward
本课程矩阵记法	(D_in,D_out)	$X @ W$
PyTorch nn.Linear	(D_out,D_in)	$X @ weight.T + bias$
不要死记形状	只要记住 PyTorch 的每一行 weight 对应一个输出 feature，它就自然是 (out_features, in_features)。	

9. 为什么必须是三套独立可学习矩阵

Q、K、V 的角色并不相同，因此让三者各自拥有 Parameter，模型就可以自己学习 “ 什么适合做匹配条件、什么适合做被匹配索引、什么适合被真正读取 ”。

$$W_Q \neq W_K \neq W_V \text{ (通常彼此独立训练)}$$

训练时它们都会通过 Loss 获得 Gradient：

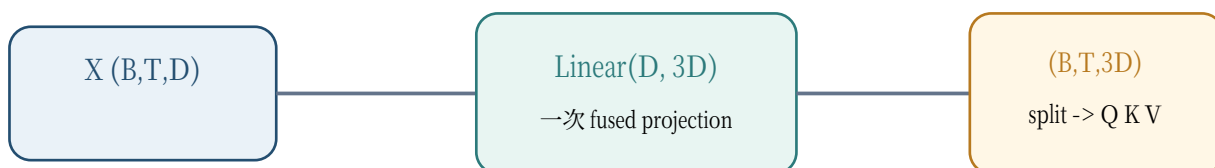
Parameter	Gradient 的来源直觉
W_Q	改变 Query -> 改变匹配 score -> 改变最终预测
W_K	改变 Key -> 改变被匹配方式 -> 改变最终预测
W_V	改变 Value -> 改变被聚合的内容 -> 改变最终预测

实验人为构造一个同时依赖 Q/K/V 的标量 Loss，Backward 后三套 Weight 都得到自己的梯度；与此同时，X 是三条分支共同的上游变量，因此 X.grad 会累加三条路径回传的贡献。

连接第 23 讲	这就是“ 分支计算图：路径内部相乘，汇合处梯度相加 ”的真实 Transformer 例子。X 同时流向 Q、K、V 三条支路，Backward 时三条支路的贡献会在 X 处汇合。
----------	--

10. 工程上常把三套 Projection 融合成一次大 Linear

三套 Linear 可以在工程上合并成一次大矩阵乘法



数学上可以分别执行三次：

$$Q=XW_Q, K=XW_K, V=XW_V$$

也可以把三套 Weight 在输出维拼起来：

$$W_{QKV} = \text{concat}(W_Q, W_K, W_V)$$

$$QKV = X @ W_{QKV}$$

然后沿最后一个维度切成 Q、K、V。实验中 fused Linear 的输出分别 split 后，与三套独立 Linear 最大误差全部为 0。

为什么工程实现
爱这样做

一次更大的矩阵乘法通常比三次小矩阵乘法更有利于硬件吞吐、kernel launch 与内存访问，因此真实 LLM 源码经常只看到一个 qkv_proj。它并不表示 Q/K/V 在数学上变成了同一个东西。

11. Q/K/V Projection 有多少参数

如果最常见地令三者总输出维都等于 D_{model} ：

W_Q, W_K, W_V each: $(D_{\text{model}}, D_{\text{model}})$

仅 Weight 参数量为：

$$3 * D_{\text{model}}^2$$

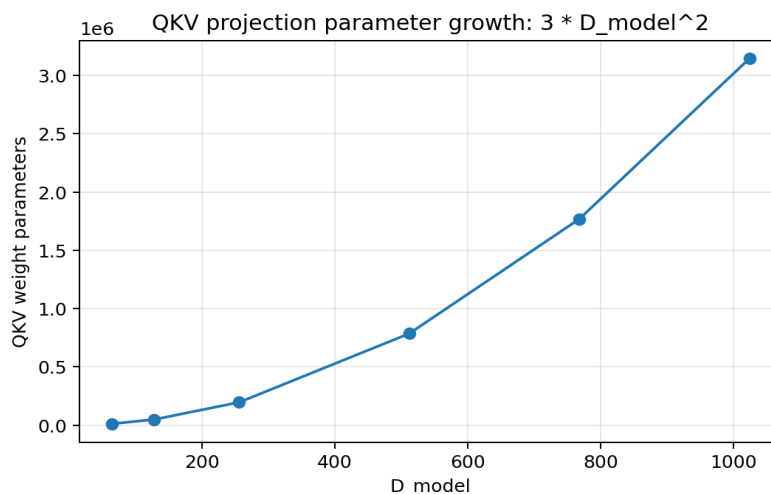
若每套 Linear 还有 bias，再增加：

$$3 * D_{\text{model}}$$

例如 $D_{\text{model}}=768$ ：

$$\text{Weight only} = 3 * 768 * 768 = 1,769,472$$

$$\text{With bias} = 1,771,776$$



参数量与序列长度的区别

QKV Projection 的 Parameter Count 主要由 D_{model} 决定，与 T 无关；但运行时计算量会随 Token 数 T 增加，因为每个 Token 都要经过这三套 Linear。

12. 到这里，Attention 还差哪一步

我们现在已经得到：

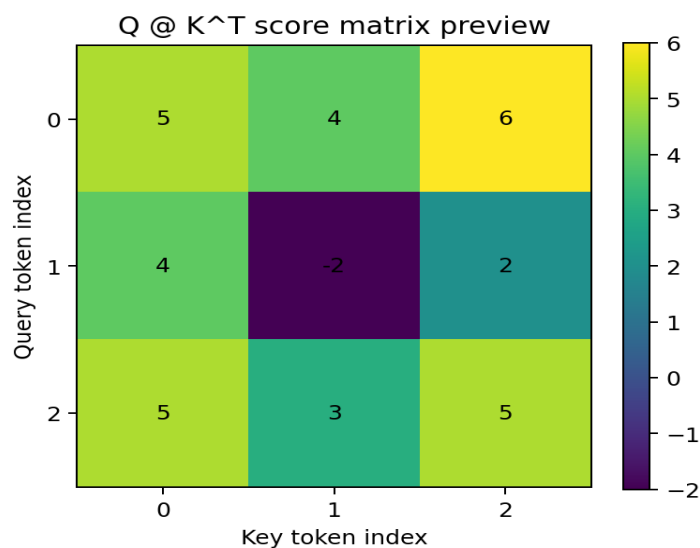
$$Q: (T, d_k), K: (T, d_k), V: (T, d_v)$$

下一步会让每一个 Query 和所有 Key 做 Dot Product：

$$\text{Scores} = Q @ K^T$$

Shape 为：

$$(T, d_k) @ (d_k, T) \rightarrow (T, T)$$



实验中的 QK^T 为：

$$[[5, 4, 6], [4, -2, 2], [5, 3, 5]]$$

这张 $T \times T$
矩阵意味着什么

行 i 表示 Query Token i ；列 j 表示 Key Token j ；元素 (i, j) 是“Token i 的 Query 与 Token j 的 Key 的匹配分数”。真正的 Attention 将在下一讲继续做 scaling、mask、softmax 和 Value aggregation。

13. Multi-Head Attention 的 Shape 预览

为了避免下一阶段突然看到更多维度，这里只做 Shape 预览，不展开 Multi-Head 的完整原理。

假设：

$$D_model = 768, num_heads = 12, d_head = 64$$

常见实现先得到总维度仍为 768 的 Q/K/V：

$$Q,K,V: (B,T,768)$$

然后 reshape / transpose 成：

$$(B, num_heads, T, d_head) = (B,12,T,64)$$

注意：Head 并不是“复制 12 份完全相同的 Attention”。每个 Head 使用自己对应的 Q/K/V 子空间，后面会独立产生 Attention Pattern。

阶段	典型 Shape
Embedding / hidden state	(B,T,D_model)
Q/K/V 总投影	(B,T,D_model)
拆 Head 后	(B,H,T,d_head)
其中	$H * d_head = D_model$
先别提前背 Multi-Head	本讲只要求理解：Q/K/V Projection 先改变 Feature Representation；拆 Head 是后续对这些 projected features 的重新组织。

14. Bias、维度和实现细节：论文里可能不完全一样

真实模型中，Q/K/V Projection 不一定都使用 bias，也不一定 d_q 、 d_k 、 d_v 与 D_{model} 完全相同。不同架构可能有不同设计。

设计项	可能的实现
Bias	有些模型使用，有些关闭
Projection	三套独立 Linear，或 fused qkv Linear
Q heads vs KV heads	后续还可能出现 MQA / GQA，使 K/V head 数少于 Q head 数
Position	有些在 Q/K 上应用 RoPE，而不是给 X 直接加 Position Embedding
科研阅读原则	不要根据变量名猜架构。直接检查 Weight Shape、reshape 过程和 forward 数据流。尤其是现代 LLM 中，GQA/MQA 会让 Q 与 K/V 的 Shape 不再完全对称。

本课程当前只建立最标准的 Self-Attention 基础。GQA/MQA 等变体以后按论文需求再深入。

15. 常见误区：这一讲最容易错在哪里

误区	正确理解
Q/K/V 是三个输入	它们通常来自同一个 X 的三套可学习投影
一得到 Q/K/V 就已经 Attention	还没有；Token mixing 要到 QK^T 与加权 V 才发生
Projection 会让 Token 互相交流	普通 Linear 只作用最后一个特征维
Q/K/V 是固定规则	$W_Q/W_K/W_V$ 都通过 Backprop 学出来
PyTorch weight 是 (in,out)	nn.Linear 保存 (out,in)
三次 Linear 与 fused Linear 数学不同	在权重正确拼接时完全等价
Q/K/V 必须维度相同	标准实现常相同，但理论与变体中可以不同
Shape Debug 最小流程	看到报错时依次写出 X、 W_Q 、 W_K 、 W_V 、Q、K、V 的 Shape；再检查矩阵乘法的中间维是否一致；最后才检查 transpose/reshape。不要靠反复试尺寸碰运气。

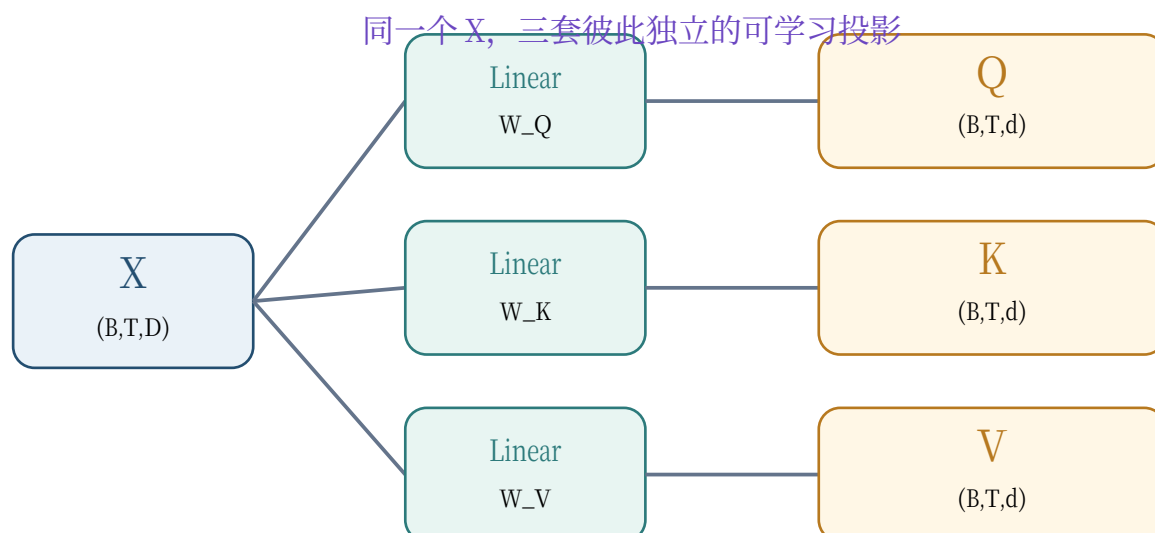
16. 与 LLM / Transformer / Agent Systems 科研的关系

你以前在 LLM 课程中已经见过 Q/K/V，但当时更多是把它们当成 Attention 的三个名词。现在应该能从深度学习底层把它们还原成可训练矩阵。

以前见过的概念	现在的底层理解
Q/K/V	X 经过三套 Linear Projection 得到的 Tensor
Attention score	Projected Q 与 K 做 Dot Product
Multi-Head	Q/K/V feature dimension 进一步拆成多个 head
RoPE	通常作用在 Q/K 的位置相关变换
LoRA	可以对 Q/K/V Projection 的 Weight 增加低秩可训练增量
Agent 的底座 LLM	Agent 每次语言理解/规划最终仍在重复这些 Tensor 运算
科研观察点	论文若声称修改 Attention 机制，先定位它到底改了 Projection、score function、normalization、mask、Value aggregation，还是 head 组织方式。把创新点放回这条数据流，机制才容易判断。

对于 Agent Systems，你通常不需要自己从零训练这些 Weight，但理解它们能帮助你判断模型改造、LoRA target modules、attention ablation、KV cache 与上下文机制相关实验到底在改什么。

17. 本讲总装图与自检



你现在应该可以从下面这一行公式一路解释到底：

$$X:(B,T,D) \rightarrow Q=XW_Q, K=XW_K, V=XW_V$$

5 分钟口述自检：

- 1) 为什么 Q 、 K 、 V 可以来自同一个 X ，却拥有不同数值？
- 2) 为什么普通 QKV Projection 不会让 Token 之间交换信息？
- 3) 如果 X 是 $(32, 128, 768)$ ， $\text{Linear}(768, 768)$ 后 Q 的 Shape 是什么？
- 4) 为什么 PyTorch 的 `q_proj.weight` 是 $(768, 768)$ ，公式却常写 XW_Q ？
- 5) 一个 fused $\text{Linear}(768, 2304)$ 为什么可以替代三套 $\text{Linear}(768, 768)$ ？
- 6) 为什么 QK^T 会得到 (T, T) ？

下一讲

第 54 讲会真正执行 Scaled Dot-Product Attention： $QK^T \rightarrow$ 除以 $\sqrt{d_k} \rightarrow$ Mask $\rightarrow$ Softmax $\rightarrow$ 乘 V 。届时第 3、4、16、17、53 讲会第一次全部汇合到同一条计算链。